CS 4400: Artificial Intelligence

Informed Search

Contents

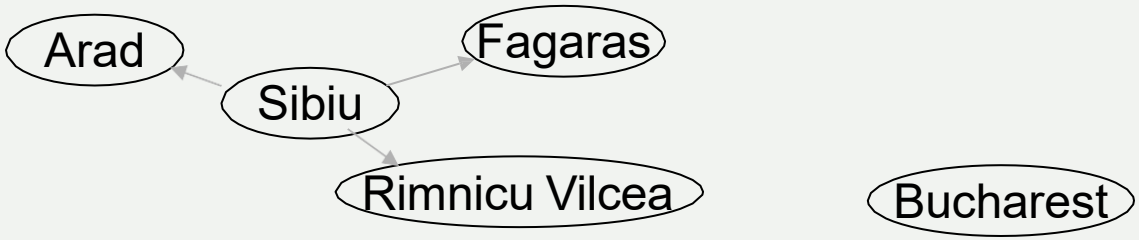
1. Informed search and heuristic function
2. Greedy best-first search
3. A* search
4. Properties of A* search
5. Memory-bounded heuristic search
6. Heuristics

Review

- Solving problem by searching
- Tree/graph search, definitions
- Uninformed searching
  - Tree-search/graph-search
  - Breadth-first search
  - Depth-first search
  - Depth-limited search
  - Iterative-deepening search
- Performance measures: completeness, optimality, space/time complexity

1/6. Informed Search

- Uninformed search used problem formulation as only information about the world
  - It can only tell if the current state is the goal or not
  - A.k.a. naïve search, brute-force search, blind search
- Problem-specific knowledge may be useful
  - The Romanian roadmap is not an arbitrary state-space graph, it is a representation of a physical map



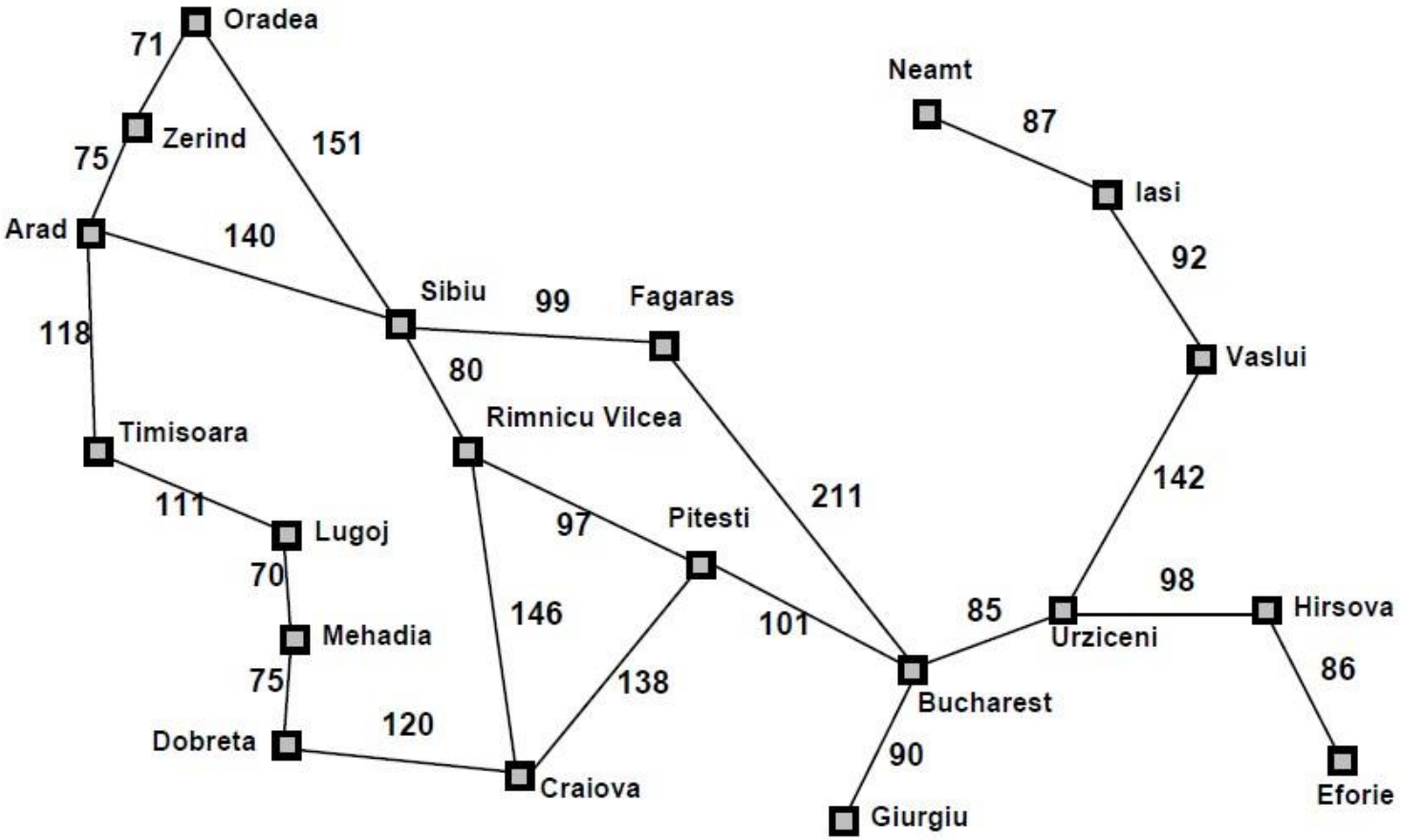
Informed search

- The strategy of a search algorithm
  - = order of choosing a node from the frontier queue
  - Priority: least depth (BFS), least path cost (UCS), most depth (DFS), etc.
  - In general, priority can be a function *f(n)*, called an evaluation function, that may be given or computable
- Evaluation function *f(n),* is an assignment of a real number to each node, and determines the search strategy
- What function should we choose for *f(n)*?

Heuristic function

- *A heuristic function h(n)*
  - *Def: An assignment of a real number to each node, such that h(n)≥0, and h(n)=0 only if n is a goal state*
  - means: estimated lowest cost from the state at node *n* to a goal state (= estimated cost-to-go)
- Use additional knowledge of the problem to determine *h(n)*
  - Example: straight-line distance from each city to Bucharest in Romania

Romania



Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

2/6. Greedy best-first search

- Use heuristic function as evaluation function
  - $f(n) = h(n)$
- Expands the node that looks most promising
  - minimum estimated distance to the goal
  - Greedy best-first search expands the node that appears to be closest to goal
- May run into dead-ends
  - Depends on what the state space looks like
- Often works well

Best-first search

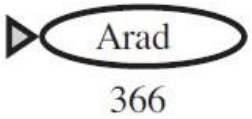
- Idea: use an evaluation function *f(n)* for each node
  - f(n) provides an estimate for the total cost.
  - → Expand the node n with smallest f(n).

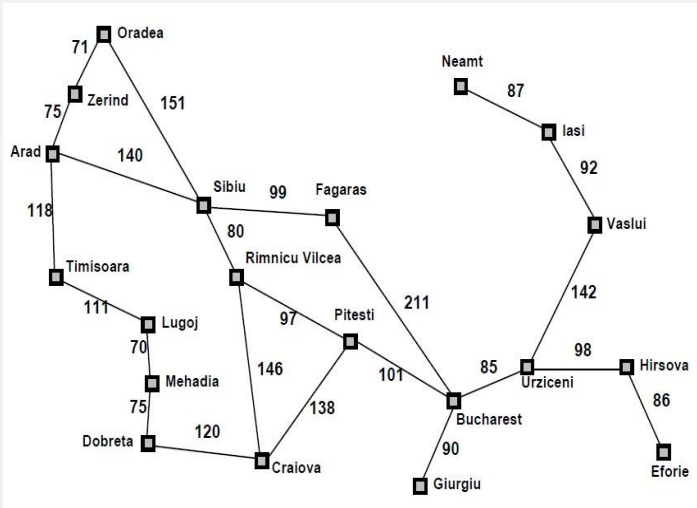
- Implementation:
  Order the nodes in Queue increasing order of cost.

- Special cases:
  - greedy best-first search
  - A* search

Example

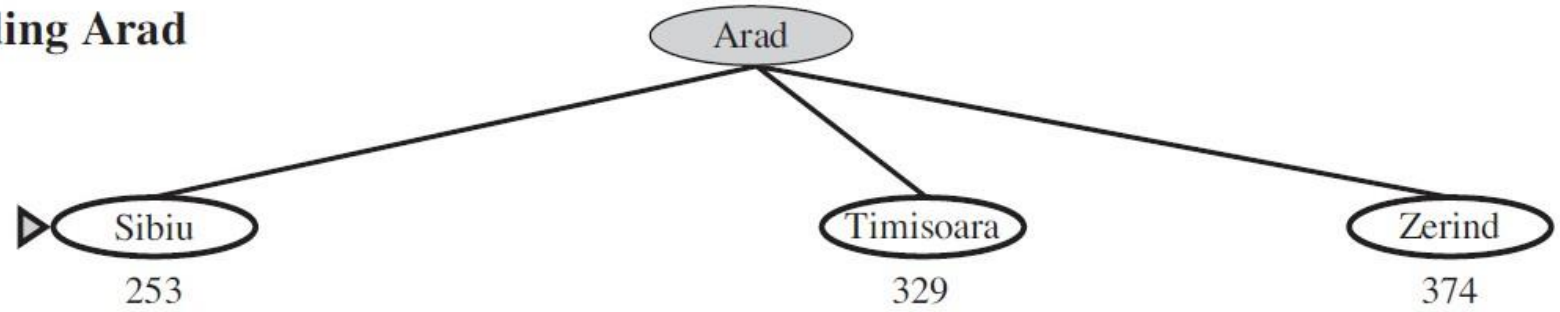
(a) The initial state

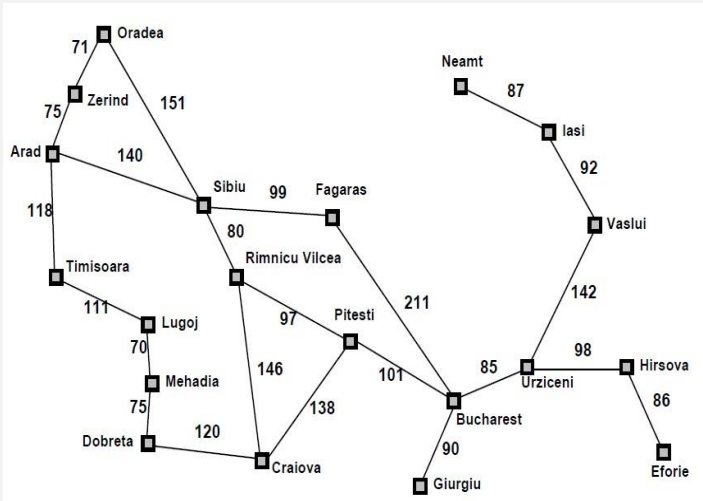




Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

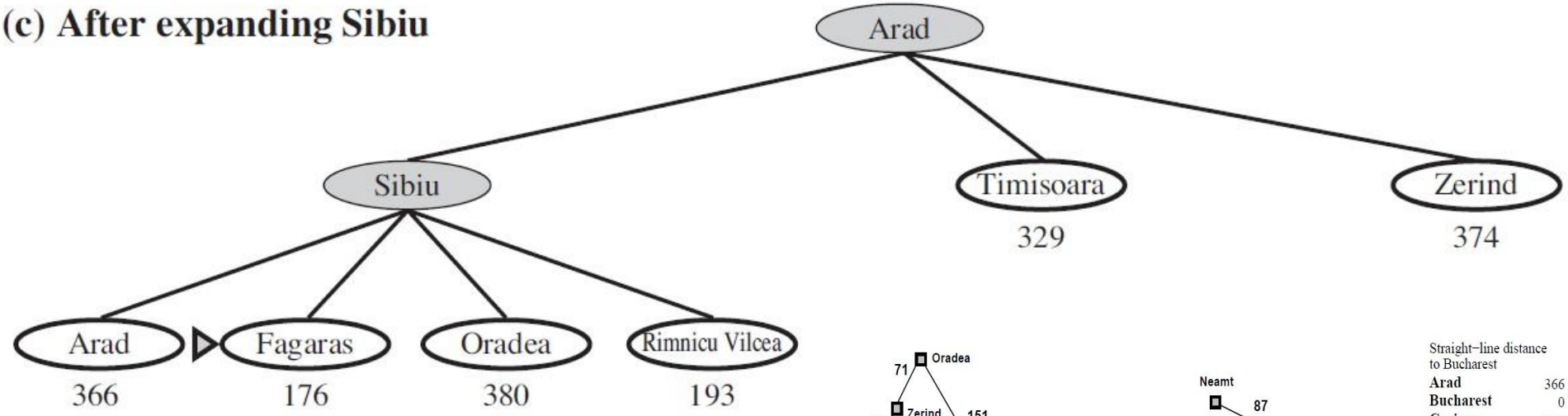
(b) After expanding Arad

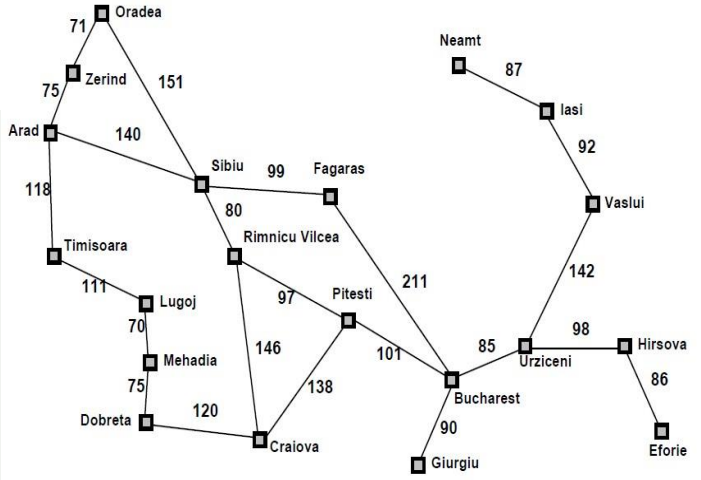




Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

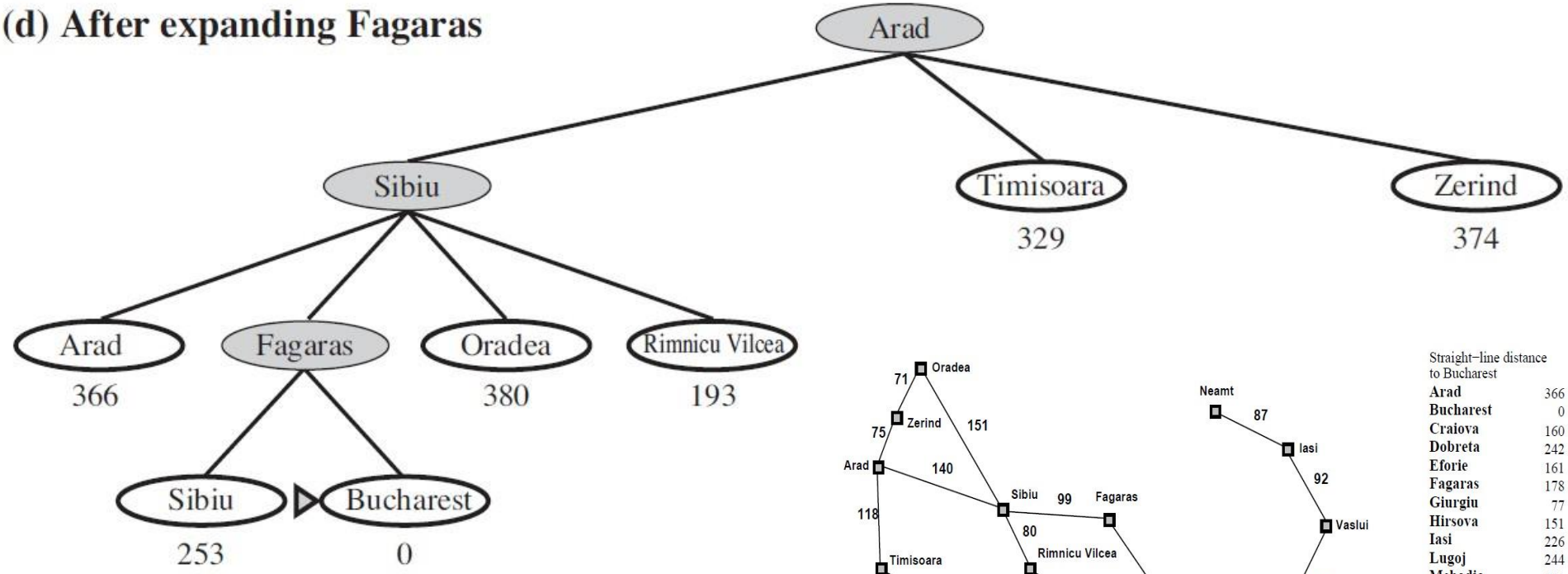
(c) After expanding Sibiu

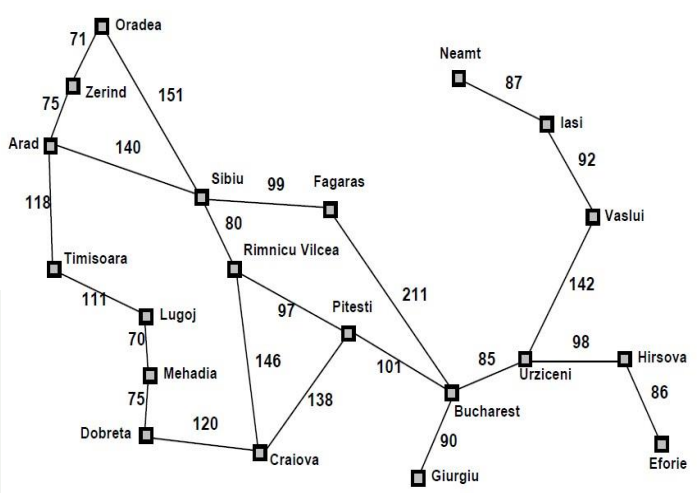




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

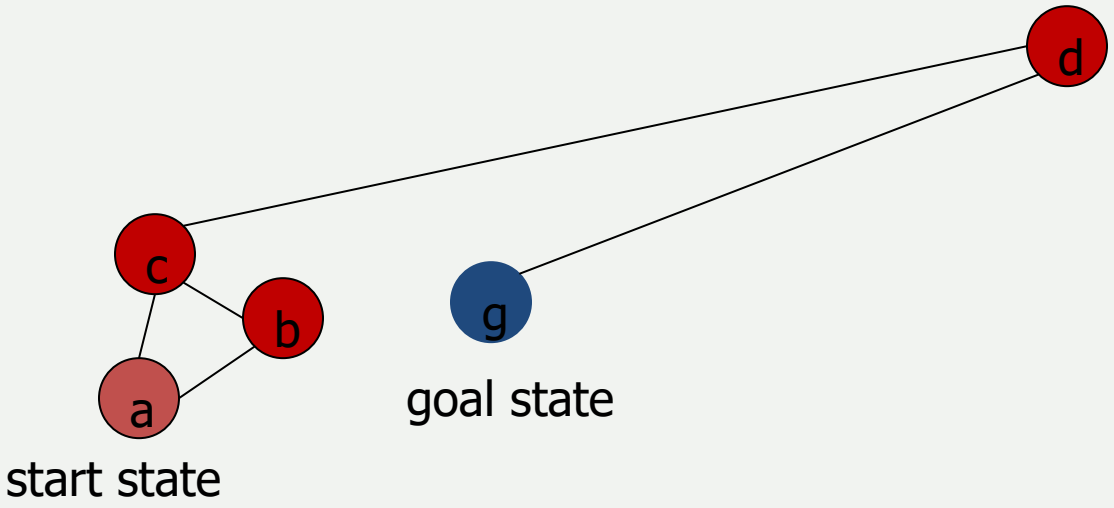
(d) After expanding Fagaras





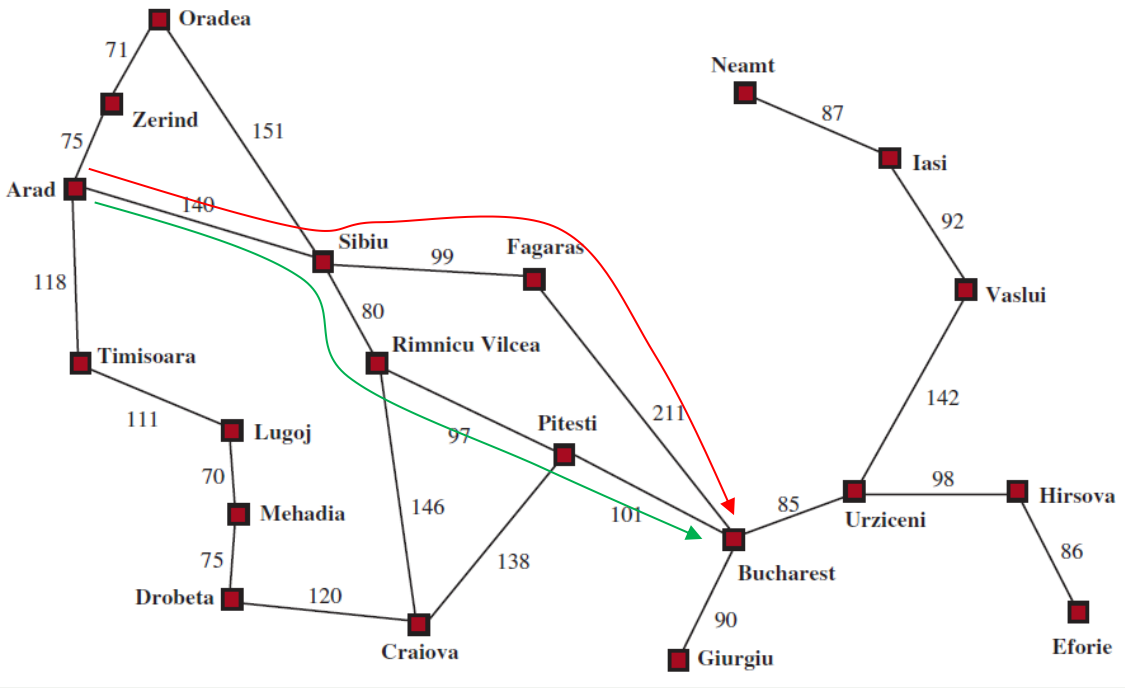
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

GBFS is not complete



f(n) = straightline distance

GBFS is not optimal



Greedy solution: Arad – Sibiu – Fagaras – Bucharest (450)

Optimal solution: Arad – Sibiu – Rimnicu Vilcea – Pitesti – Bucharest (418)

This greedy strategy does *not* account for the cost to get to the current state.

Properties of greedy best-first search

- Complete? No – can get stuck in loops.
- Time? $O(b^m)$, but a good heuristic can give dramatic improvement
- Space? $O(b^m)$ - keeps all nodes in memory
- Optimal? No

  e.g. Arad→Sibiu→Rimnicu
  Virea→Pitesti→Bucharest is shorter!

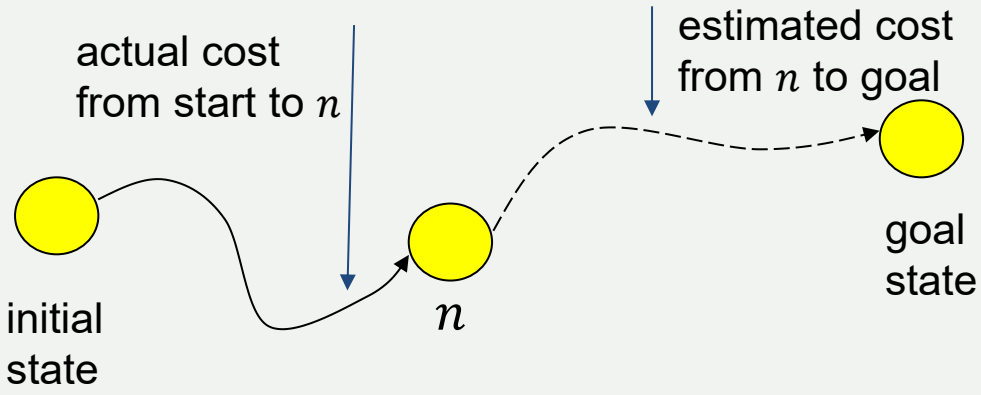
3/6. A* Search

- Incorporates both path cost and heuristic cost
  - Evaluation function for UCS = cost up to now
    - $f(n) = g(n)$: path cost
  - Evaluation function for GBFS = estimated cost to go
    - $f(n) = h(n)$: estimated cost-to-go
    - A node with the least estimated cost to go may have a high path cost leading to the node
  - A* search takes both into account
    - $f(n) = g(n) + h(n)$: path cost + estimated cost-to-go

3/6. A* Search

P. E. Hart, N. J. Nilsson, and B. Raphael (1968)

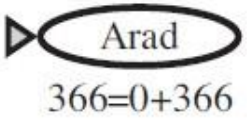
$$f(n) = g(n) + h(n)$$

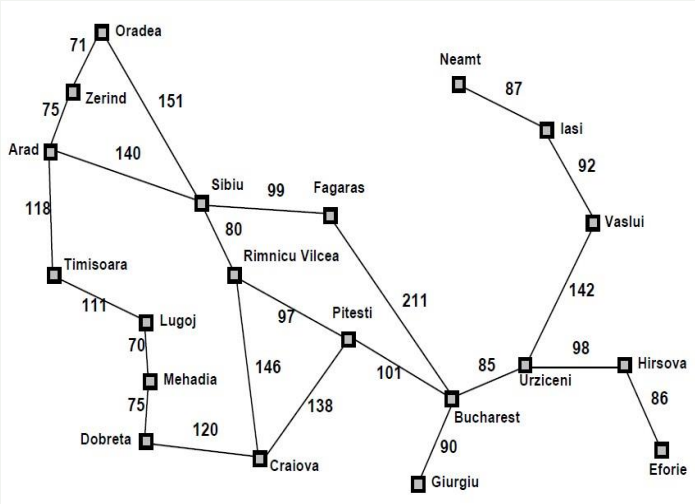


$f(n)$: estimated cost of the best path that continues from n to a goal.

Example: Romanian map

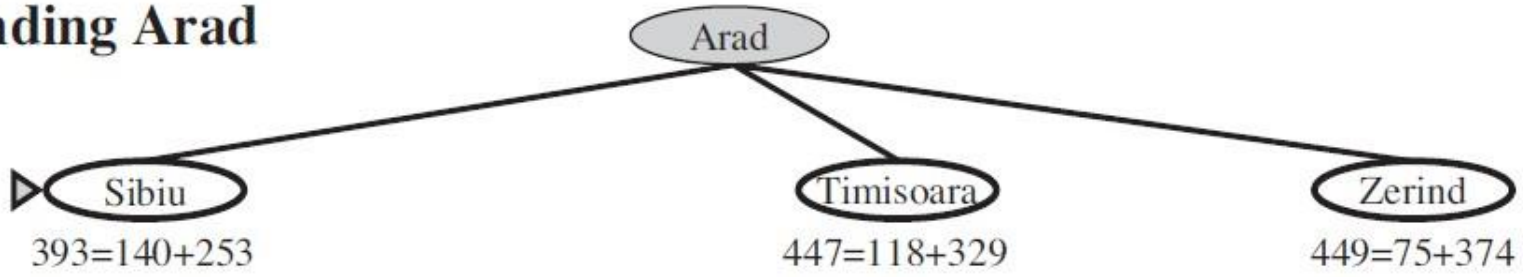
(a) The initial state

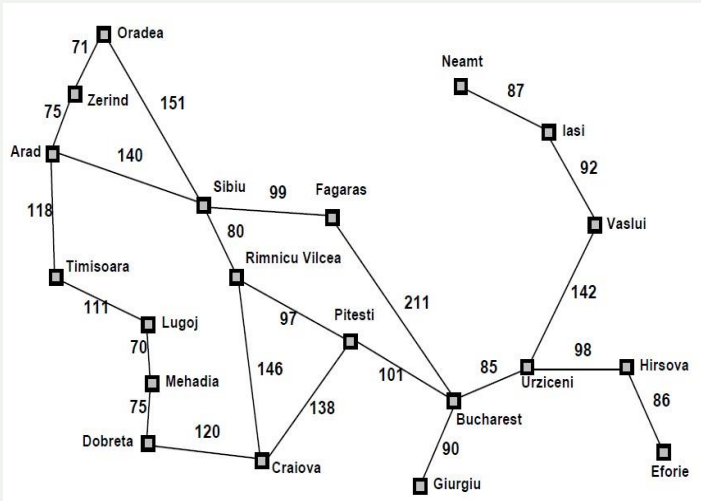




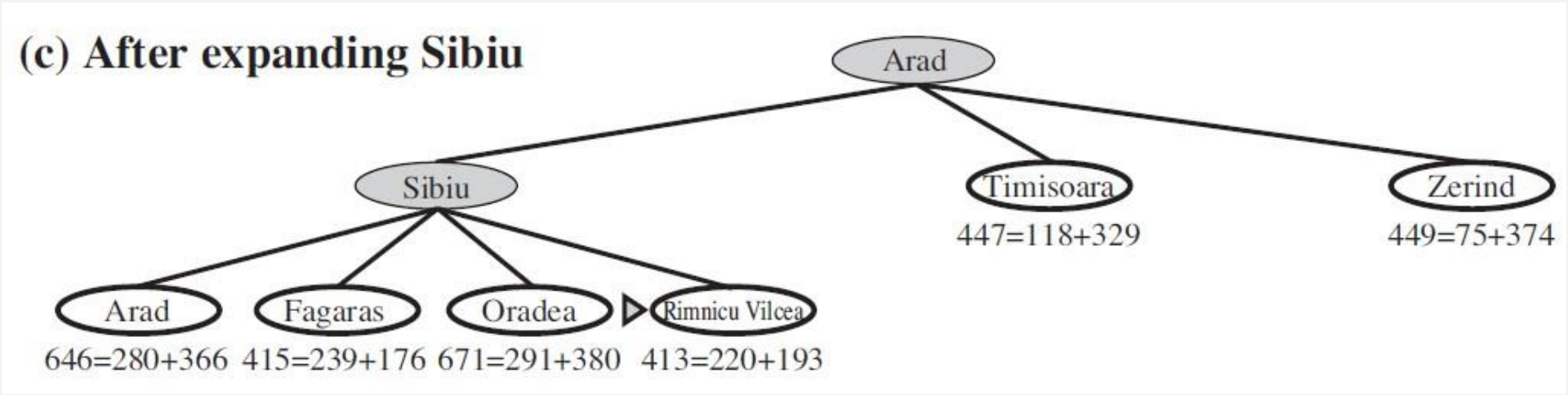
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

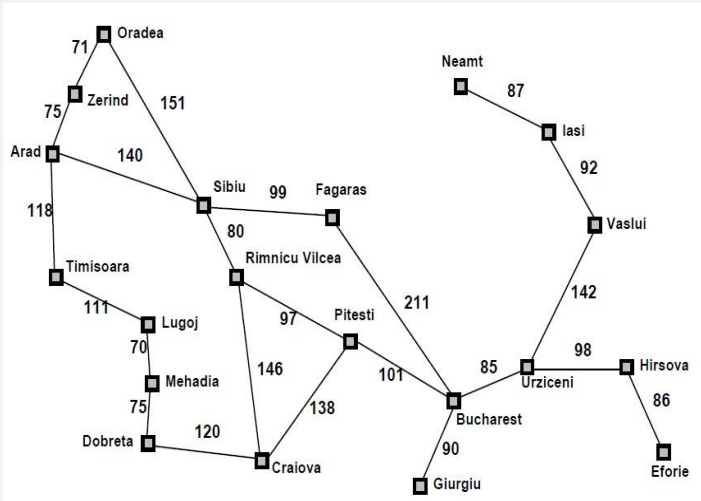
(b) After expanding Arad





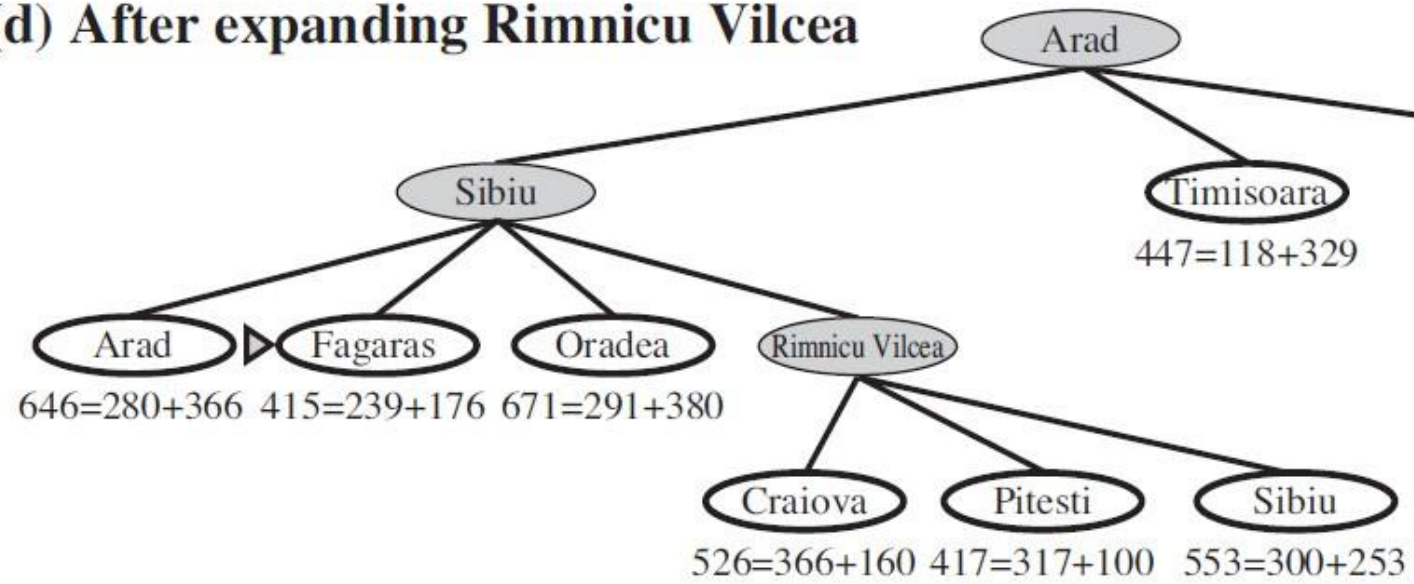
Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

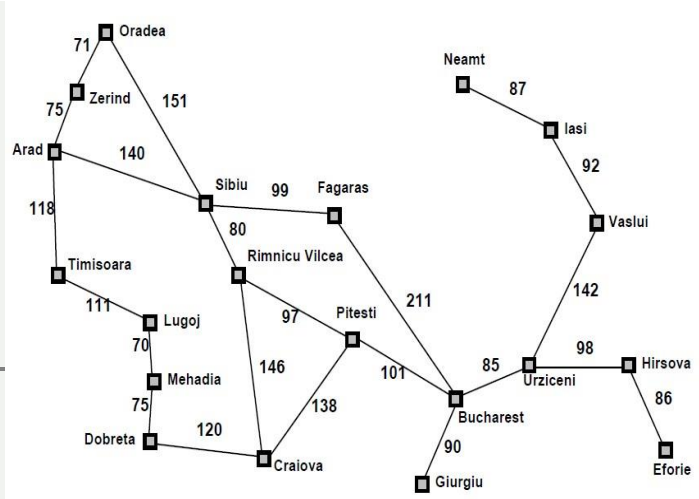




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

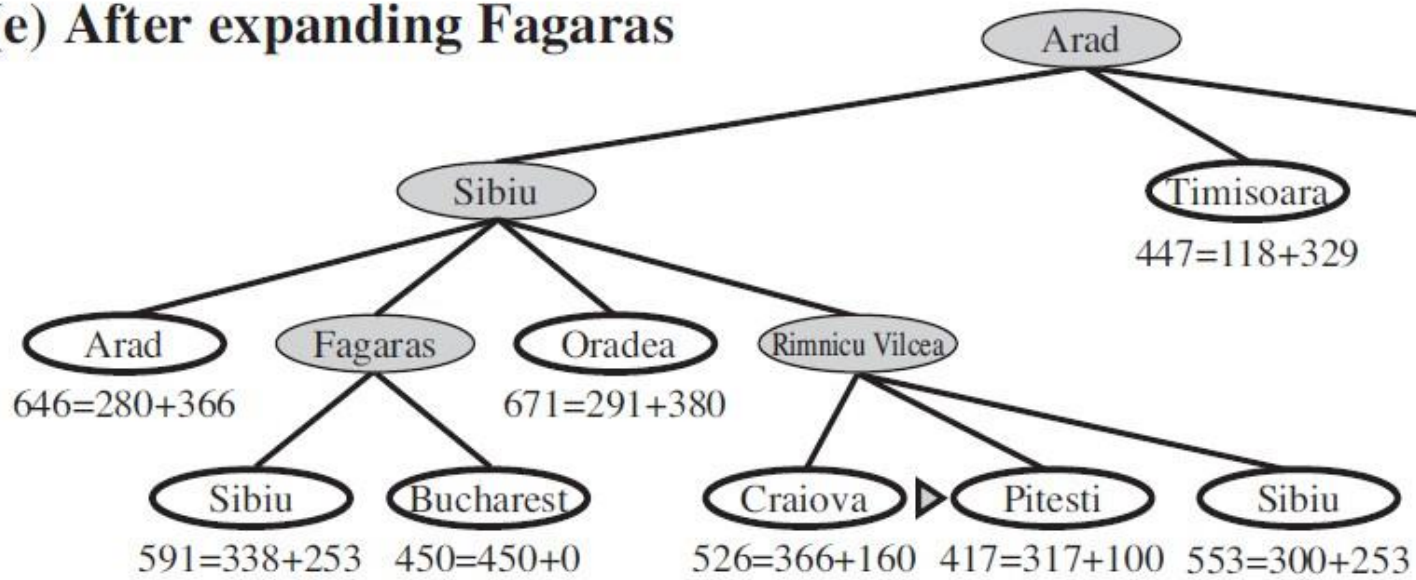
(d) After expanding Rimnicu Vilcea

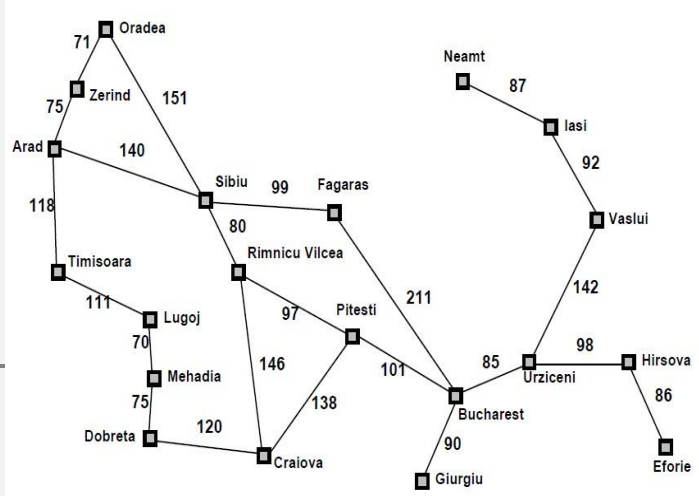




Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

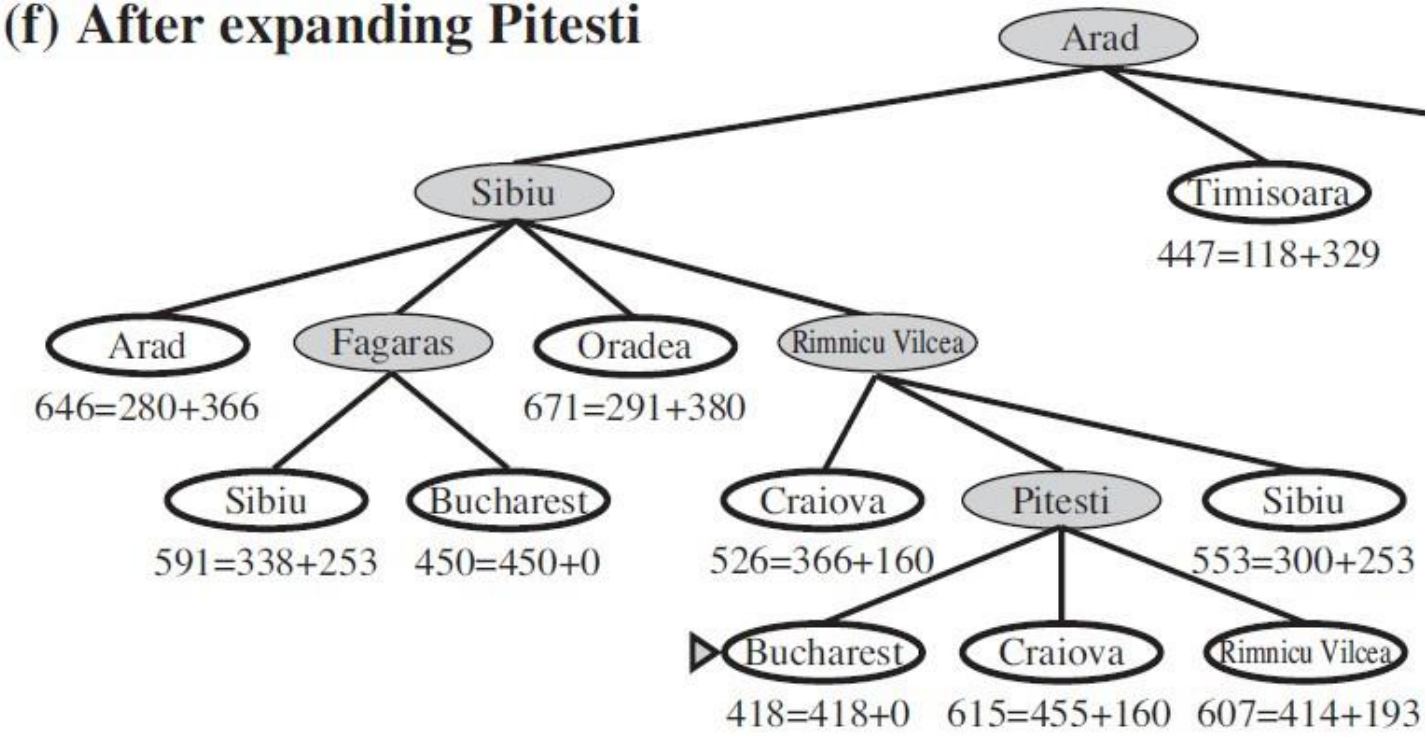
(e) After expanding Fagaras

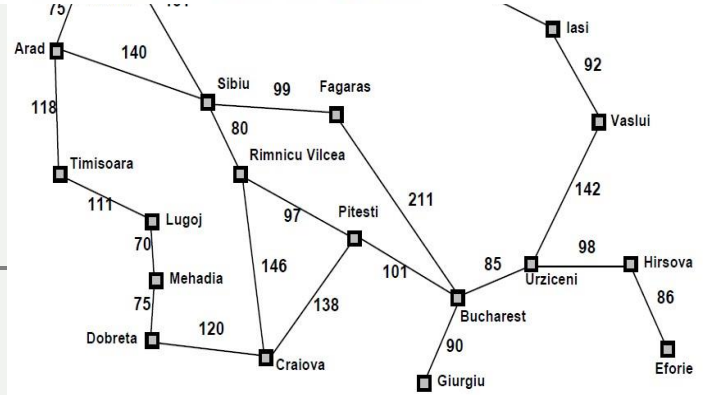




Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

(f) After expanding Pitesti





Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

4/6. Properties of A* search

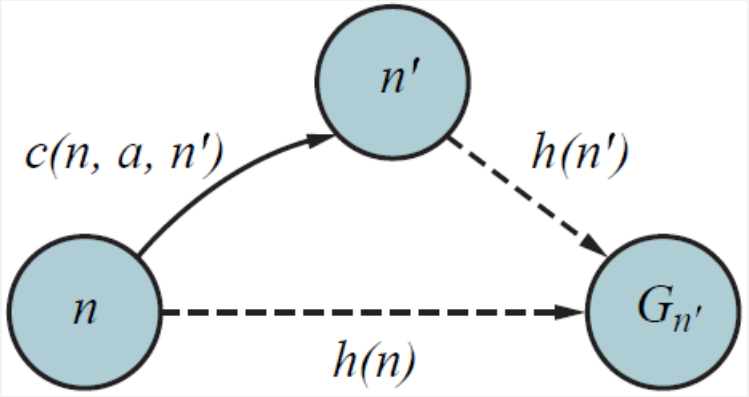
- A* search is complete and optimal if
  - *h(n)* is admissible (for tree search)
  - *h(n)* is consistent (for graph search)
- *Def: h(n)* admissible if
  - *h(n)* always *underestimates* the cost to the goal
  - i.e. $h(n) \leq h^*(n)$ for all *n*, where $h^*(n)$ is the (true) least cost to the Goal
    - An admissible heuristic never overestimates the cost to reach the goal, i.e., it is optimistic
- *Def: h(n)* consistent if
  - $h(n) \leq c(n,a,n') + h(n')$, where *n'* is a child of *n*
  - *~ triangle inequality*
- If *h(n)* consistent then *h(n)* admissible

Consistent heuristics

A heuristic h is *consistent* if for every two nodes n and n' such that n' is generated from n by some action a, the following inequality holds:

$$h(n) \leq c(n, a, n') + h(n')$$

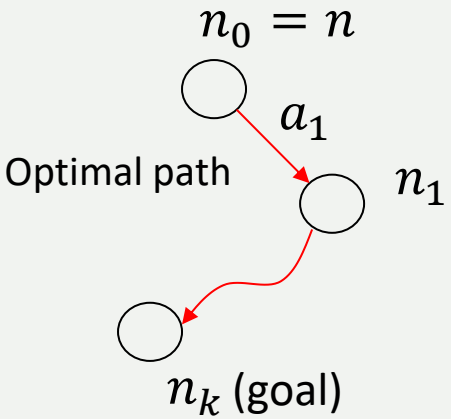
(*triangle inequality*)



- Every consistent heuristic is admissible.

Consider the optimal path $n_0 = n, n_1, \ldots, n_k$ (goal).

$$\begin{aligned} h(n) = h(n_0) &\leq c(n_0, a_1, n_1) + h(n_1) \\ &\leq c(n_0, a_1, n_1) + c(n_1, a_2, n_2) + h(n_2) \\ &\vdots \\ &\leq \sum_{i=1}^{k} c(n_{i-1}, a_i, n_i) + \underbrace{h(n_k)}_{=0} = h^*(n) \end{aligned}$$

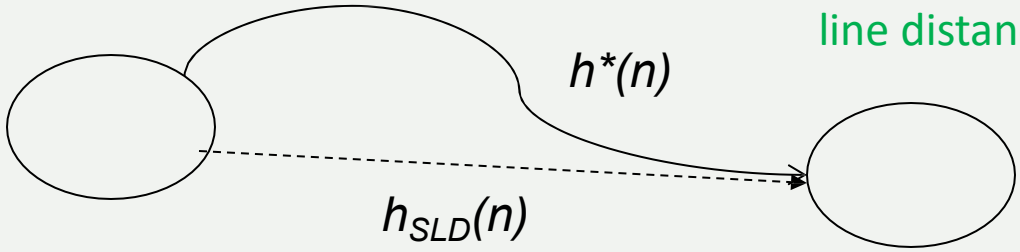


- A* with a consistent heuristic is cost-optimal.

Straight line distance heuristic

- In Romanian navigation problem, is the straight line distance heuristic *admissible*?
  - Yes
  - Why? Straight line distance is the shortest distance between two points
  - $h_{SLD}(n) \leq h^*(n)$ for all n

h_{SLD} is admissible because the actual distance to Bucharest cannot be less than the straight-line distance.



Proof of the optimality of A* - 1

- Let's prove that A* is optimal if *h(n)* is consistent for graph search
- Lemma 1: if *h(n)* is consistent, then *f(n)* along any path is nondecreasing

- Proof:
  - Let *n'* is a child of *n*, then *g(n')* = *g(n)* + *c(n,a,n')* for some action *a*, and we have
  - $f(n') = g(n') + h(n') = g(n) + c(n,a,n') + h(n')$

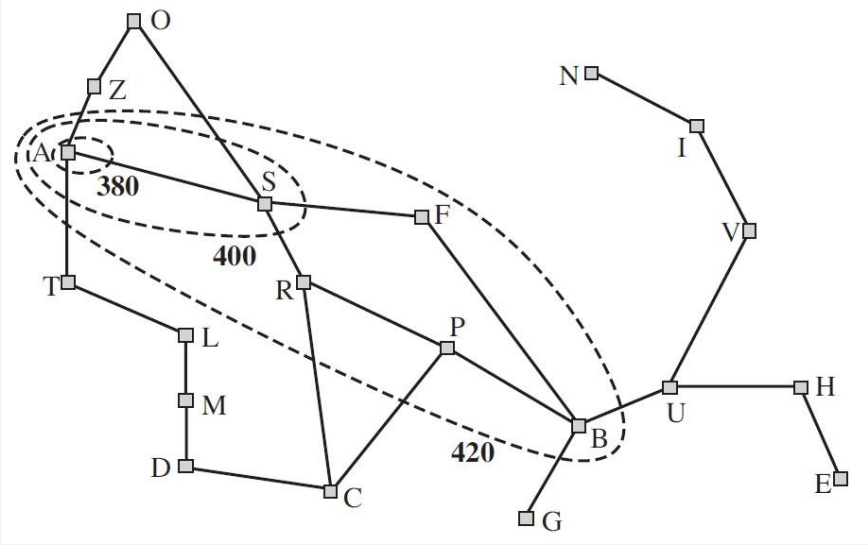
    $\geq g(n) + h(n) = f(n)$

Proof of the optimality of A* - 2

- Whenever a node is selected for expansion, the optimal path to that node has been found by A*
- Proof:
  - Suppose the lemma were not true, i.e., then there is a frontier node *n'* on the optimal path from the root to *n*
  - By lemma 1, $f(n') < f(n)$, and *n'* must have been selected before *n*

    → Contradiction!
- As a result, if a goal is found, then the current path to the goal is an optimal one

Visualization: search contour



- Nodes expanded in A* have increasing *f(n)'s*
  - Search "contour"
  - All nodes inside the contour have smaller f and are expanded
- Good heuristic creates "narrow" contours
  - cf. concentric circles for uniform-cost search

Properties of A* search

- Effect of narrow contours: A* search "prunes" subtrees (e.g., Timisoara) that are unlikely to be the optimal path

- Time and space complexity
  - Depends on the heuristic function
  - Time complexity is exponential in *relative error x depth*
  - Space complexity is worse: keep all nodes in memory
  - For a practical use, memory usage need to be bounded

Iterative deepening A* search

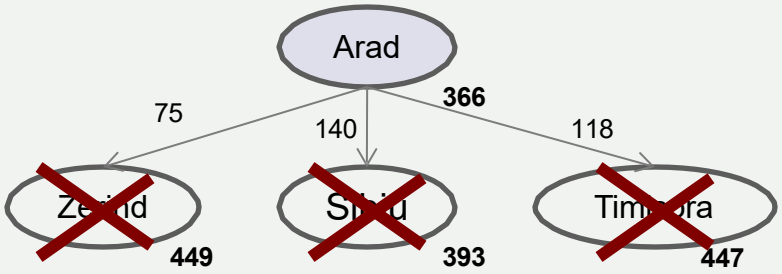
- A situation somewhat similar to DFS
  - DFS: linear space complexity, but not optimal
  - Cure: from DFS to IDDFS
- Iterative deepening A* search
  - Perform A* with a cutoff on *f(n)*
  - Increase cutoff to the next smallest *f(n)* from all nodes encountered during search
  - Repeat

IDA*



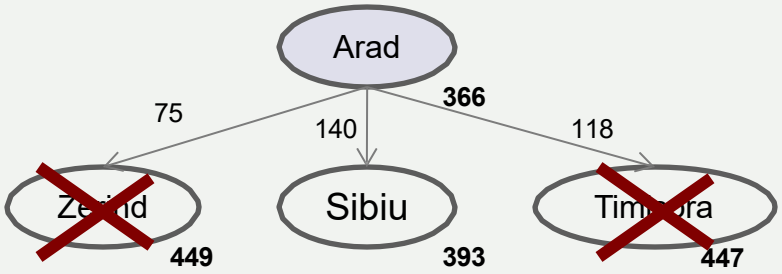
$f_{max}=366$

IDA*



$f_{max}=366$

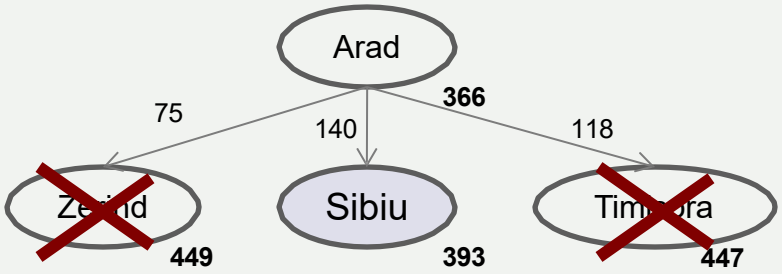
IDA*



~~$f_{max}=366$~~

$f_{max}=393$

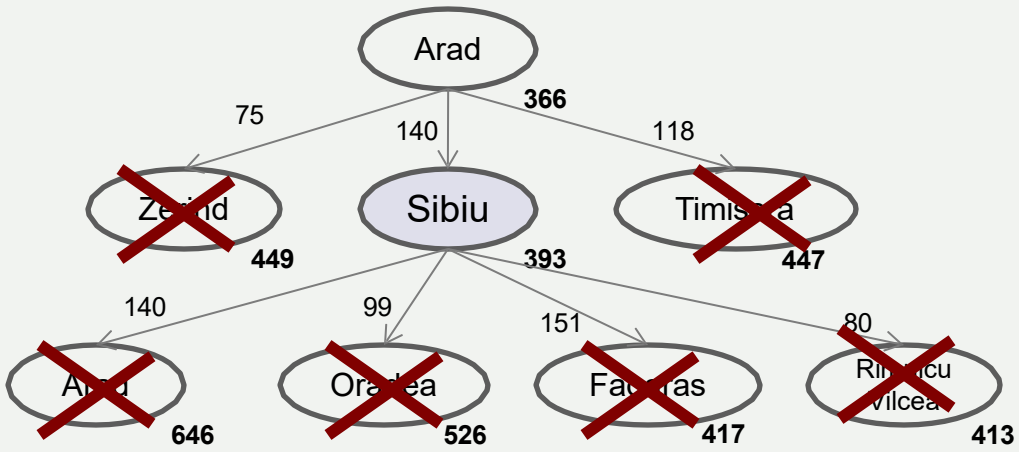
IDA*



~~$f_{max}=366$~~

$f_{max}=393$

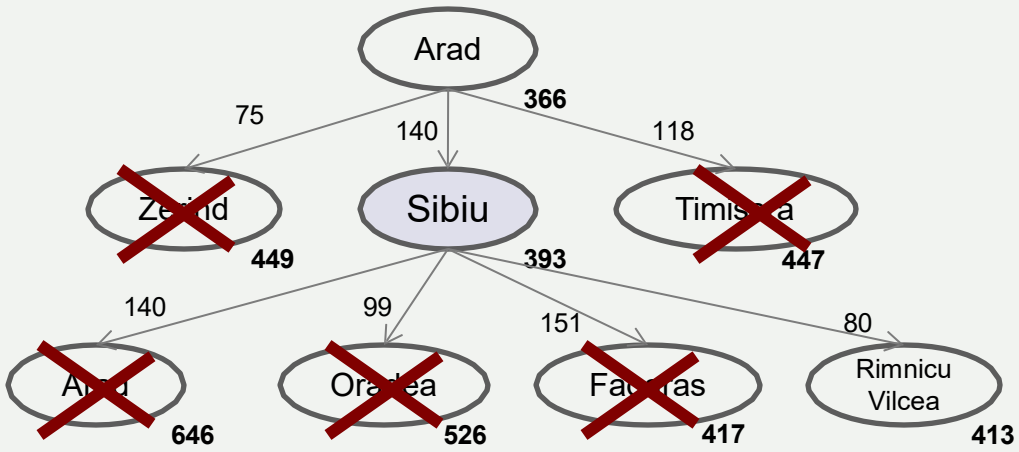
IDA*



f_{max}=~~366~~

f_{max}=393

IDA*

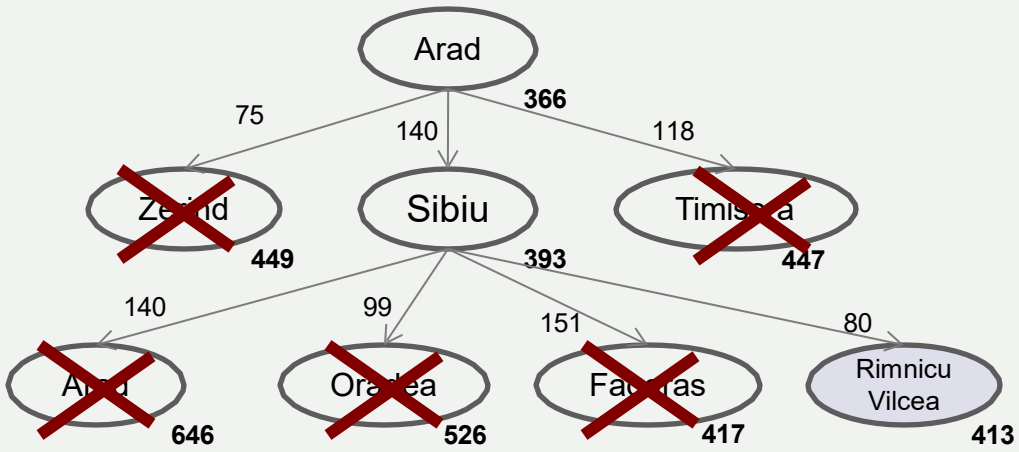


f_{max}=366

f_{max}=393

f_{max}=413

IDA*

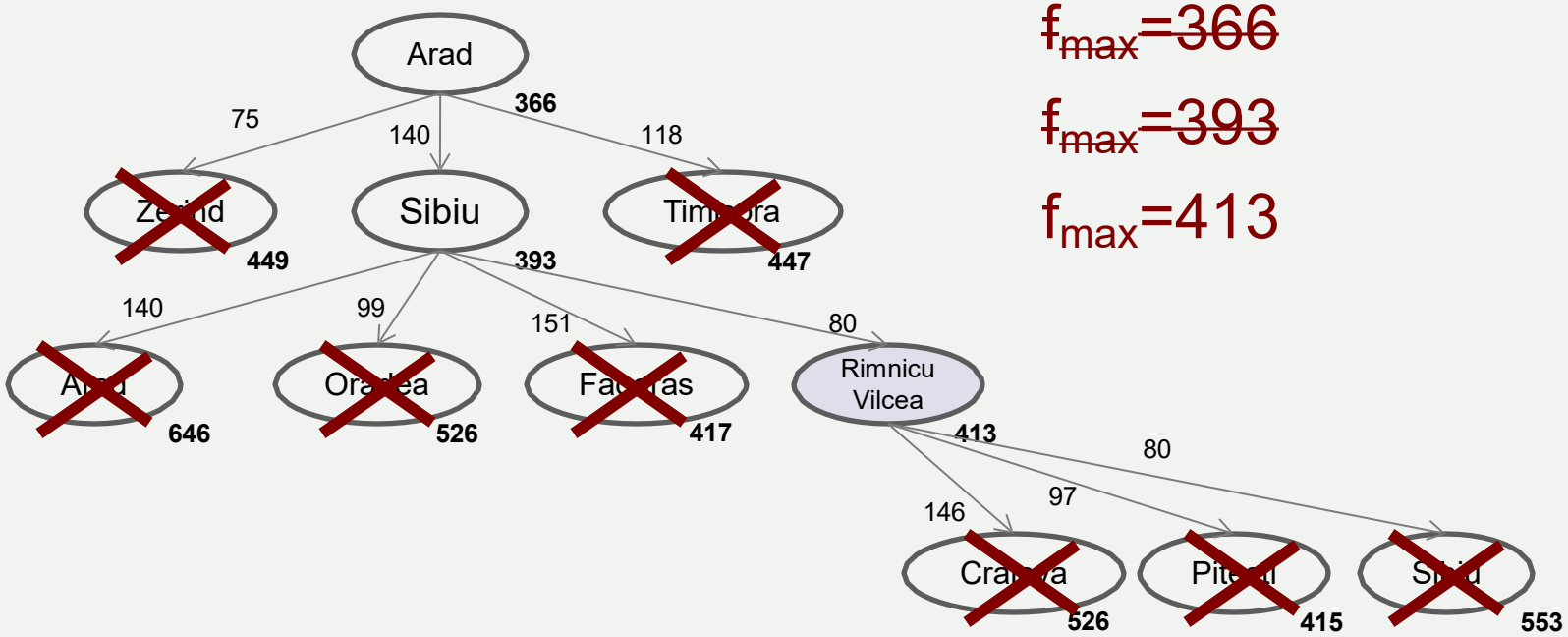


~~$f_{max}=366$~~

~~$f_{max}=393$~~

$f_{max}=413$

IDA*

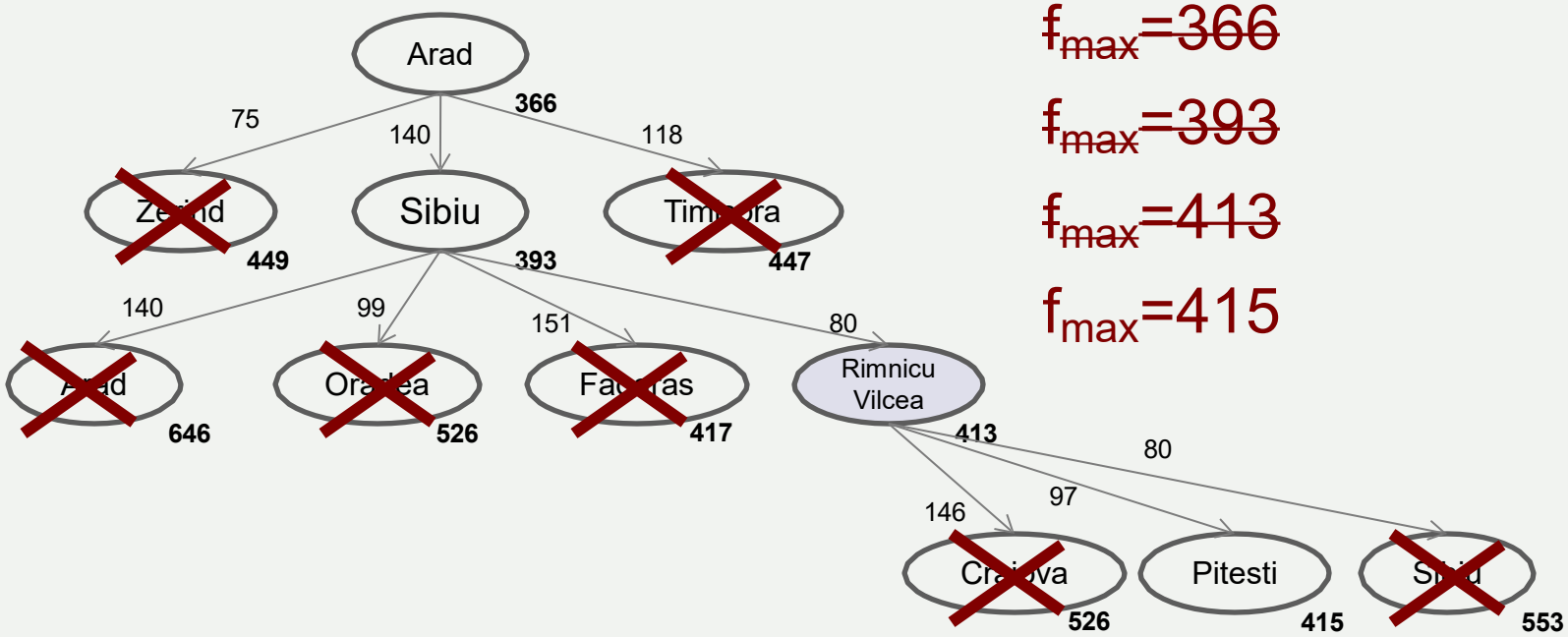


f_{max}=~~366~~

f_{max}=~~393~~

f_{max}=413

IDA*



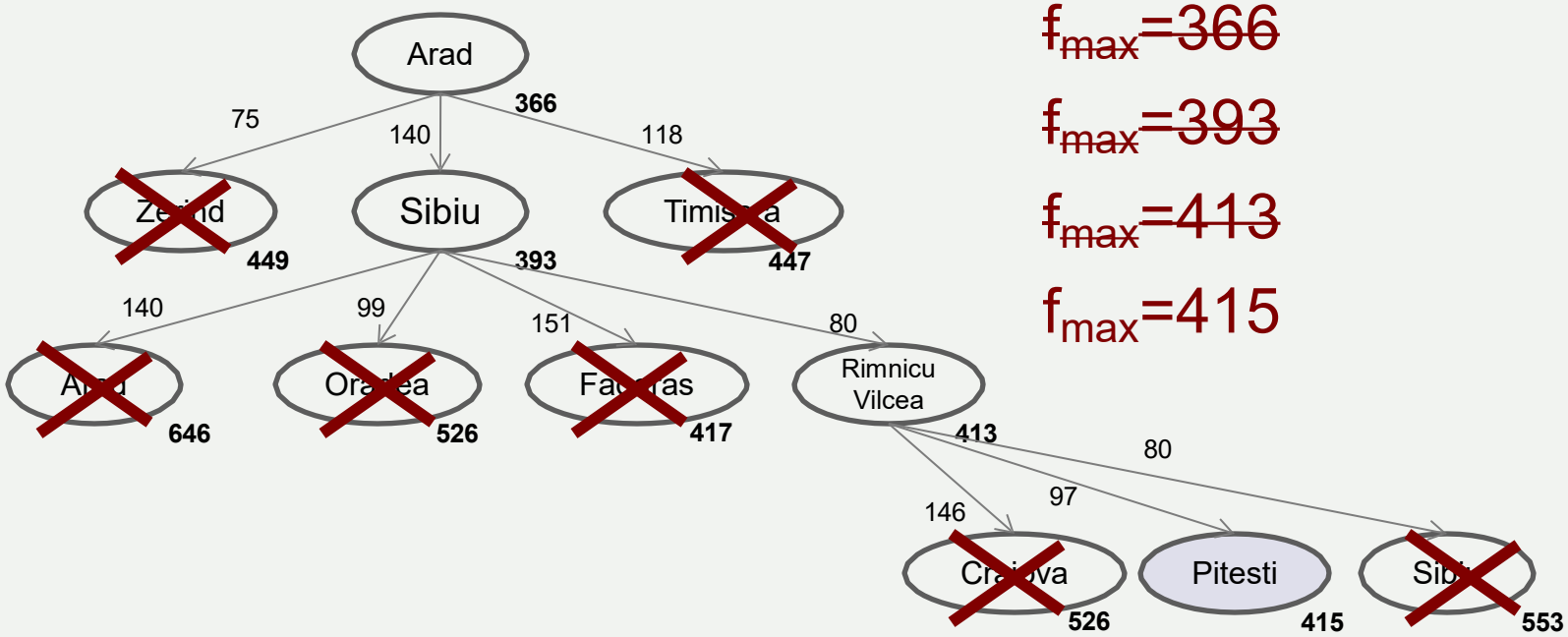
f_{max}=~~366~~

f_{max}=~~393~~

f_{max}=~~413~~

f_{max}=415

IDA*



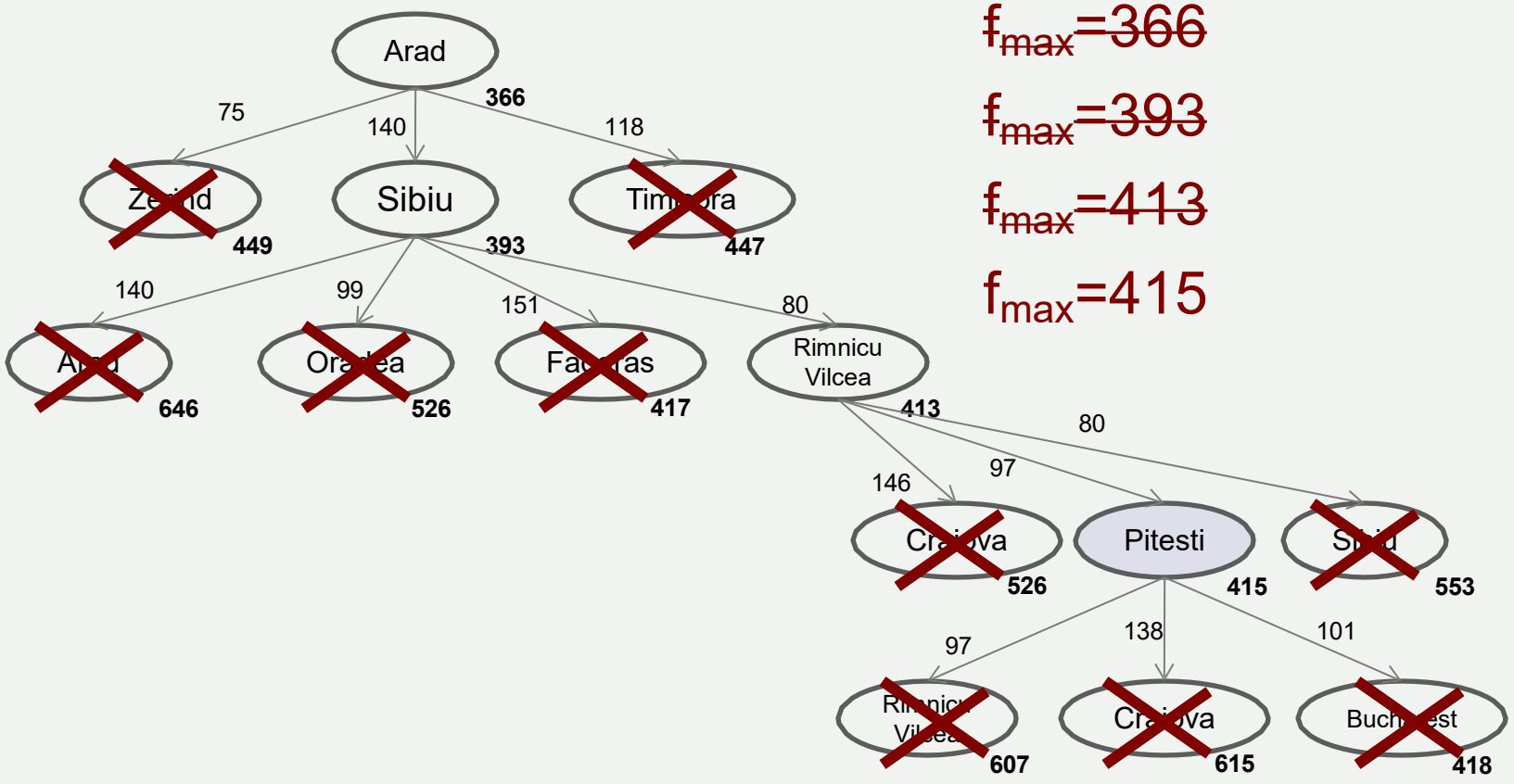
f_{max}=~~366~~

f_{max}=~~393~~

f_{max}=~~413~~

f_{max}=415

IDA*



~~$f_{max}=366$~~

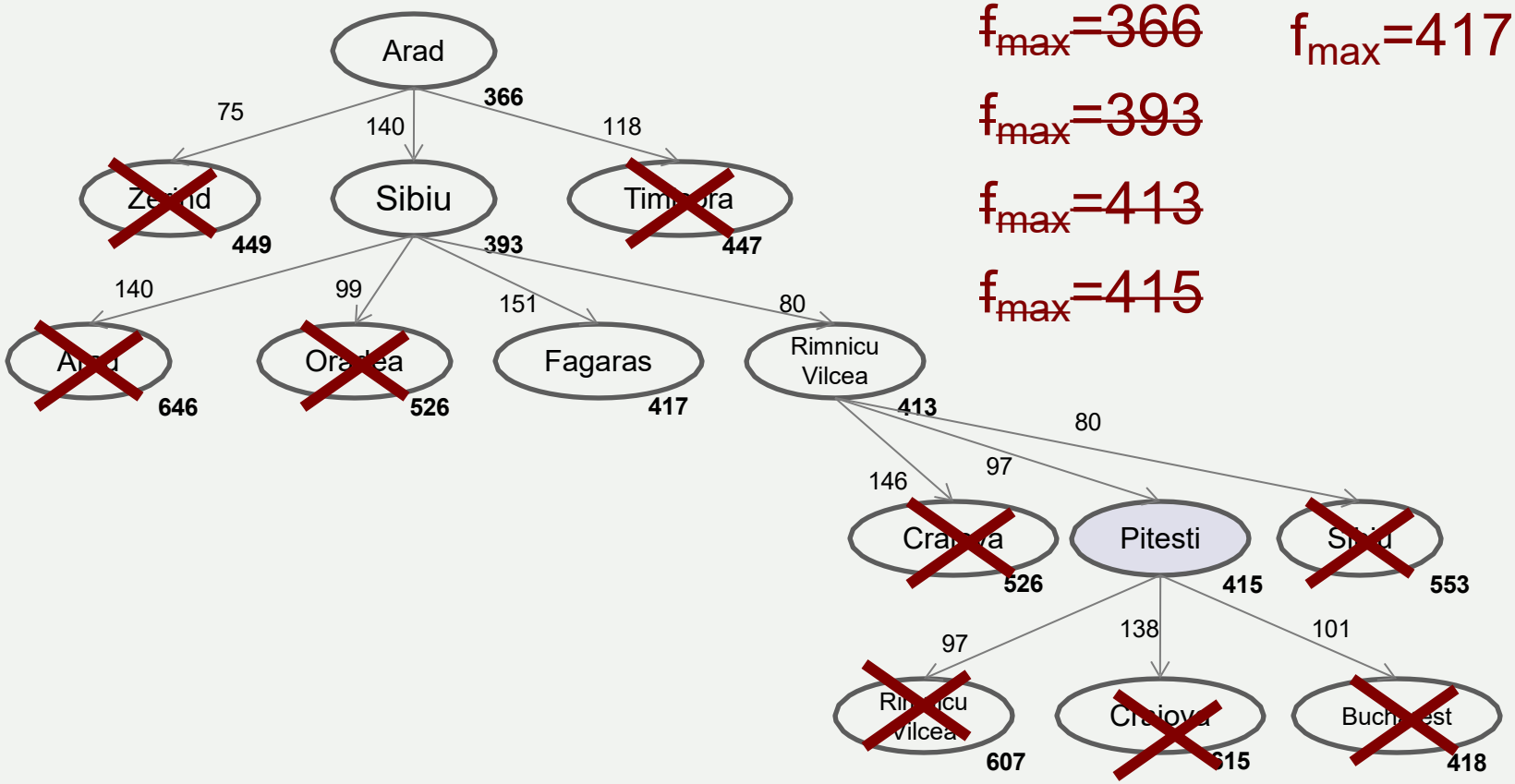
~~$f_{max}=393$~~

~~$f_{max}=413$~~

$f_{max}=415$

What is f_{max} now?

IDA*



f_{max}=~~366~~

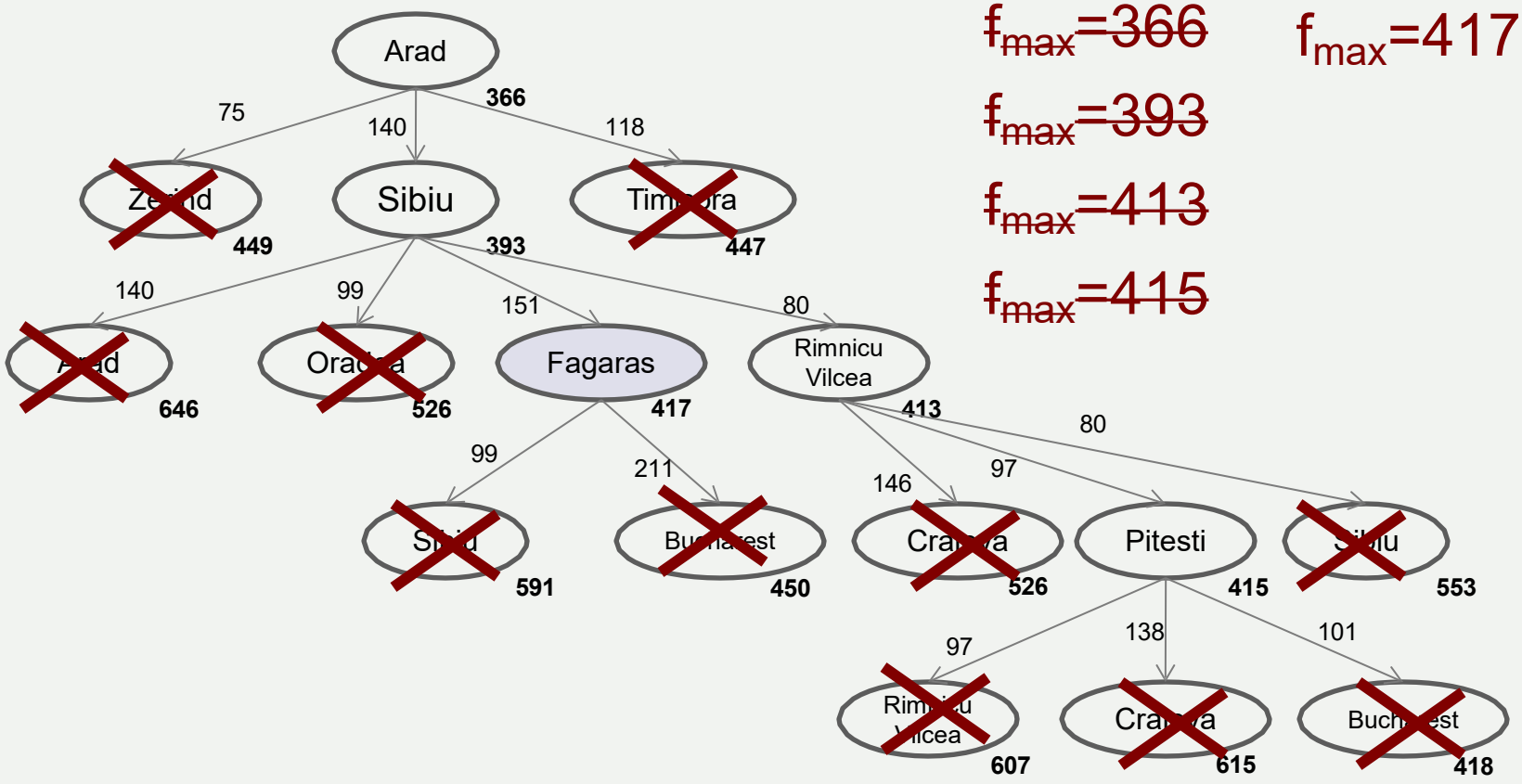
f_{max}=~~393~~

f_{max}=~~413~~

f_{max}=~~415~~

f_{max}=417

IDA*



f_{max}=~~366~~

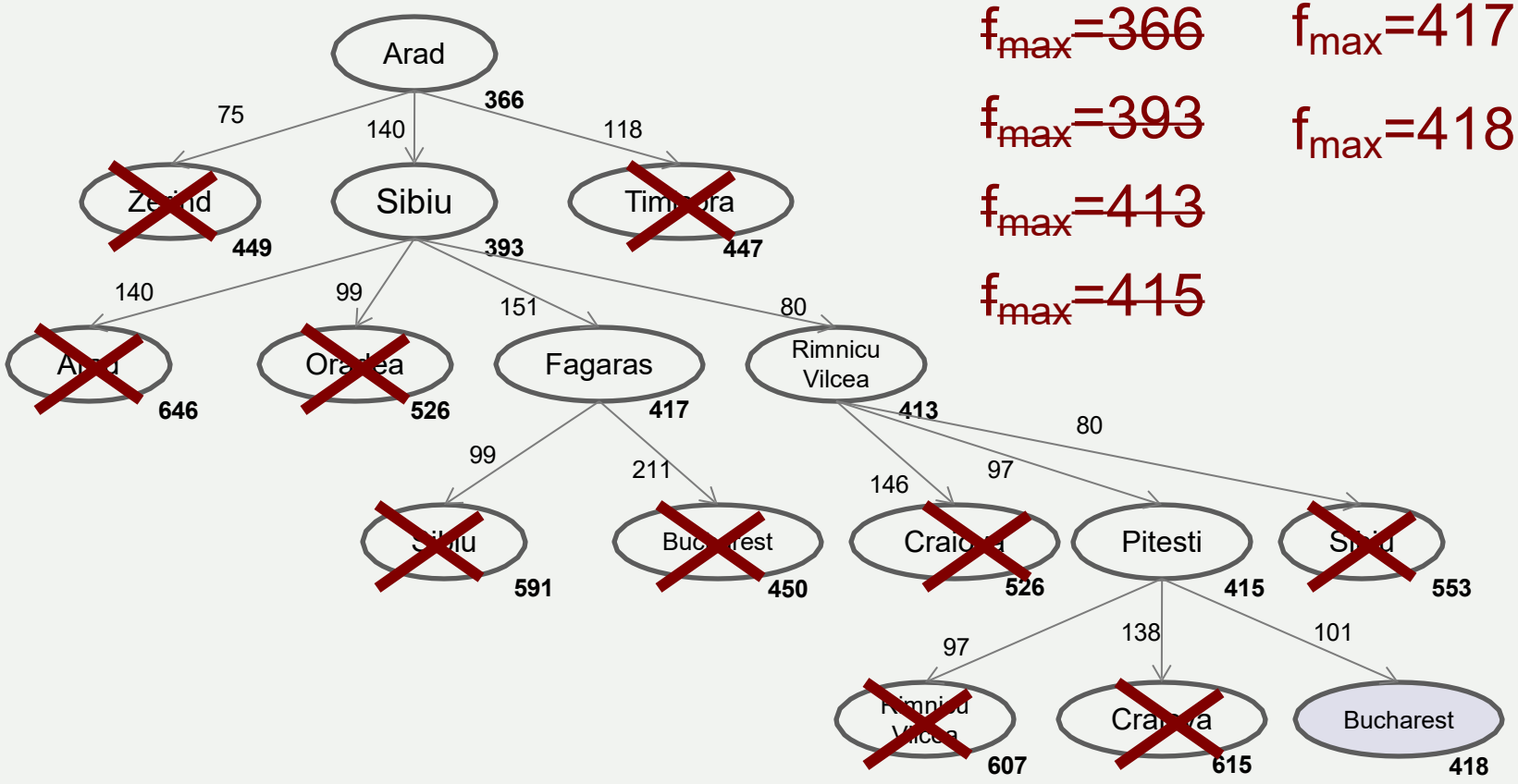
f_{max}=~~393~~

f_{max}=~~413~~

f_{max}=~~415~~

f_{max}=417

IDA*



f_{max}=~~366~~

f_{max}=~~393~~

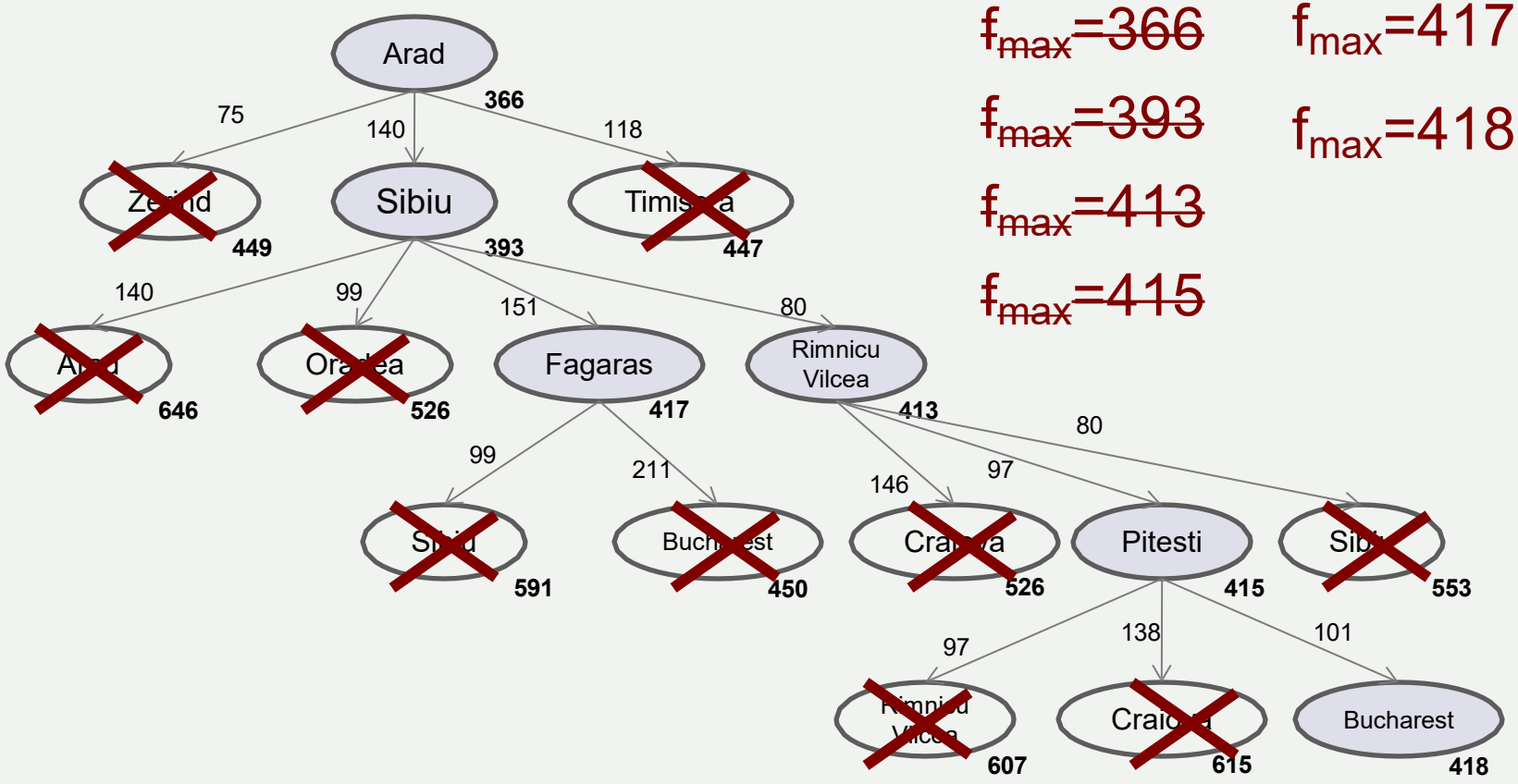
f_{max}=~~413~~

f_{max}=~~415~~

f_{max}=417

f_{max}=418

IDA*



f_{max}=~~366~~

f_{max}=~~393~~

f_{max}=~~413~~

f_{max}=~~415~~

f_{max}=417

f_{max}=418

Memory-bounded heuristic search 2

- Recursive Best First Search (RBFS)
  - Like DFS, only stop when your current path costs more than some alternative
  - *Each node keeps* f_{limit} kept at each node
  - If search is stopped, update *f*-costs of parents with children's scores

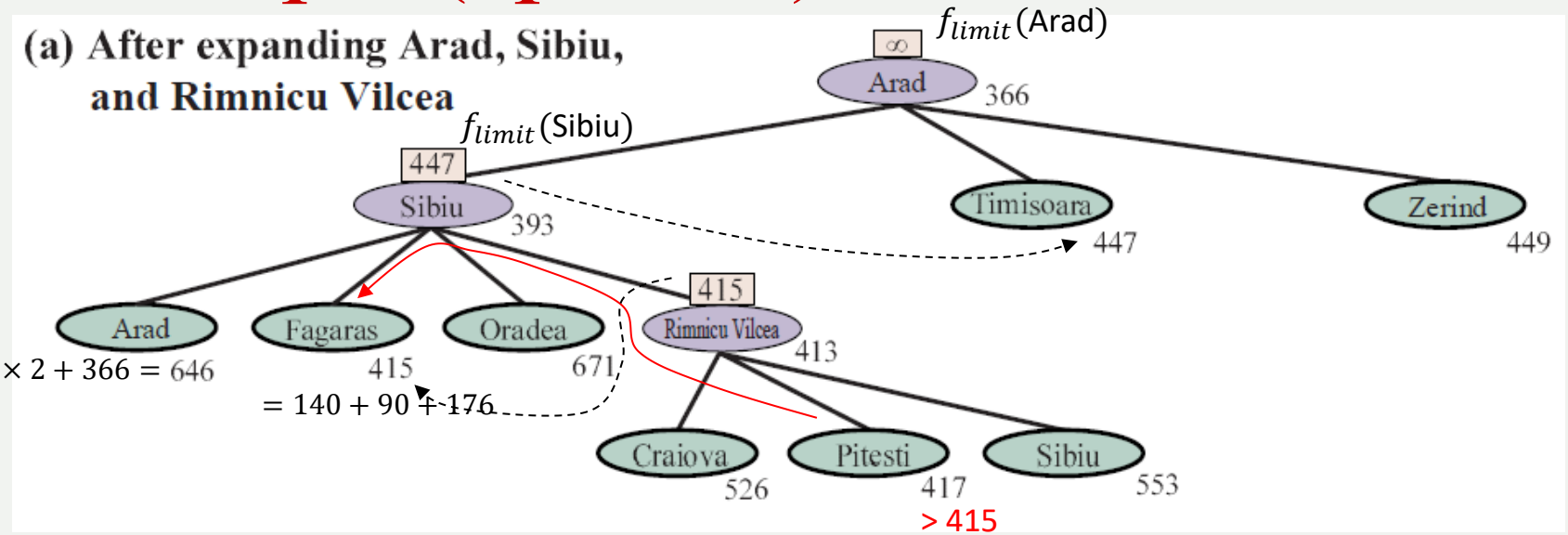
- (Simple) Memory bounded A*: MA*, SMA*
  - When out of memory, throw out node with *highest f*-cost

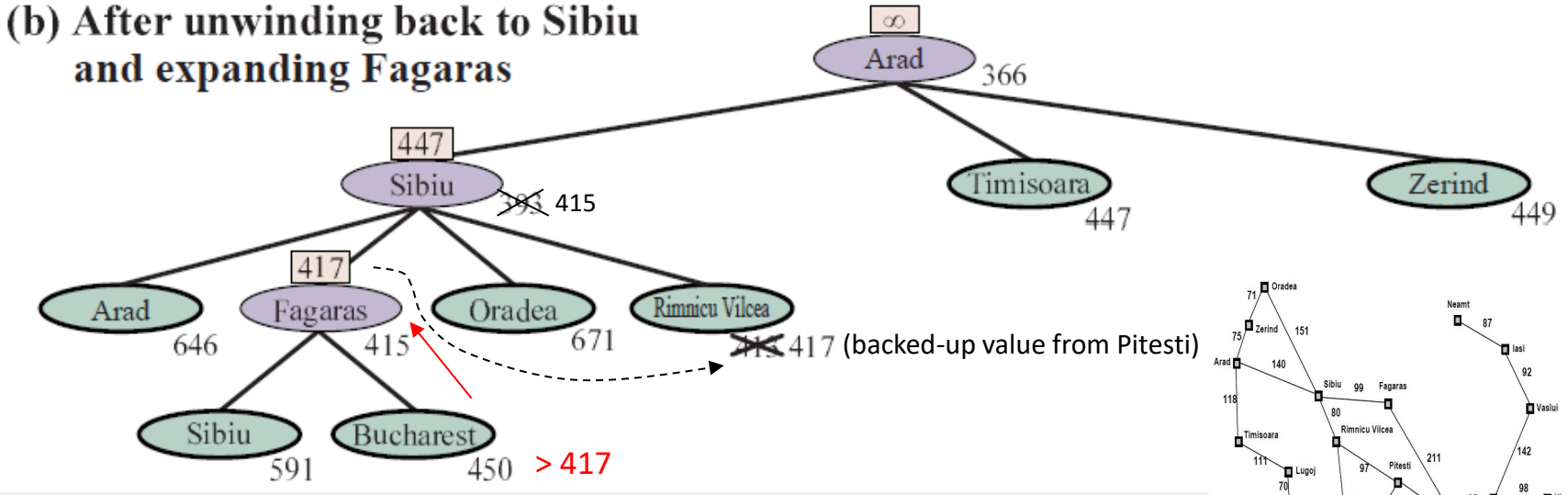
Recursive best-first search (optional)

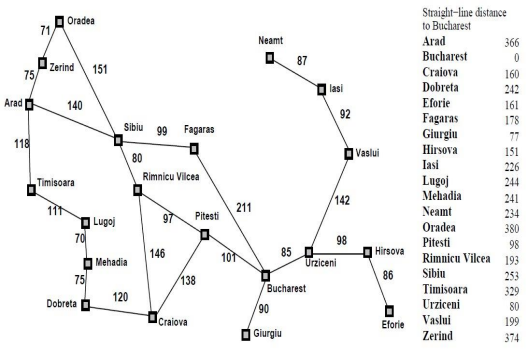
```
function RECURSIVE-BEST-FIRST-SEARCH(problem) returns a solution, or failure
    return RBFS(problem, MAKE-NODE(problem.INITIAL-STATE), ∞)

function RBFS(problem, node, f_limit) returns a solution, or failure and a new f-cost limit
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  successors ← [ ]
  for each action in problem.ACTIONS(node.STATE) do
      add CHILD-NODE(problem, node, action) into successors
  if successors is empty then return failure, ∞
  for each s in successors do /* update f with value from previous search, if any */
      s.f ← max(s.g + s.h, node.f))
  loop do
      best ← the lowest f-value node in successors
      if best.f > f_limit then return failure, best.f
      alternative ← the second-lowest f-value among successors
      result, best.f ← RBFS(problem, best, min(f_limit, alternative))
      if result ≠ failure then return result
```

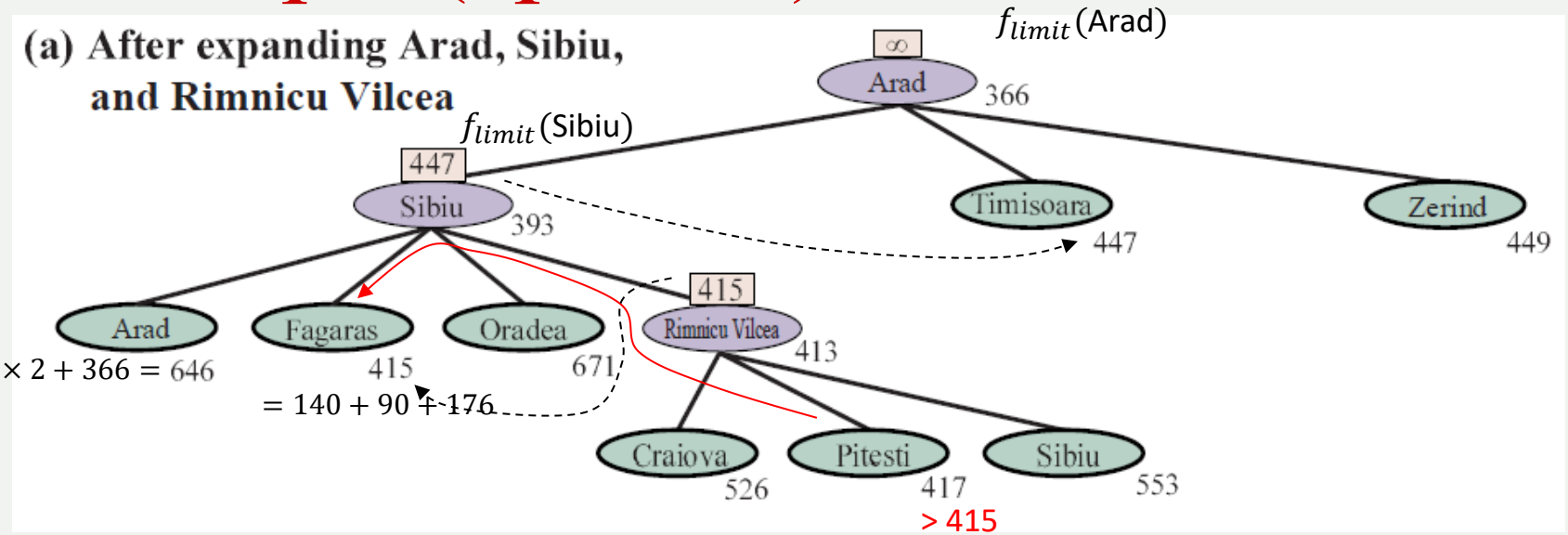
Example (optional)

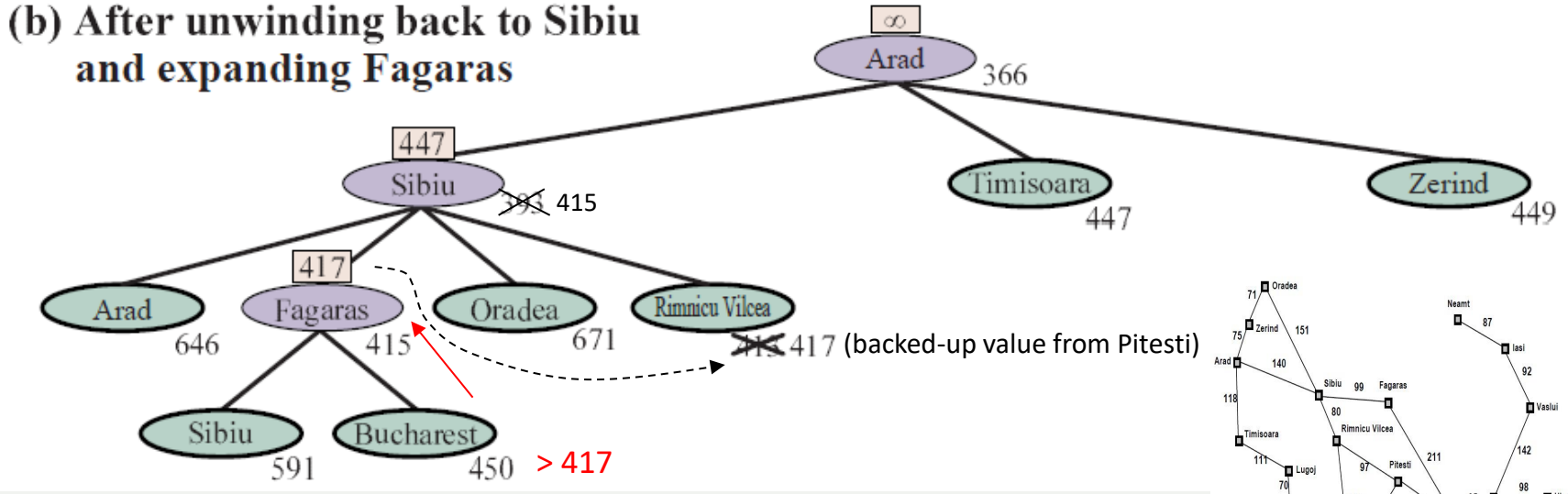


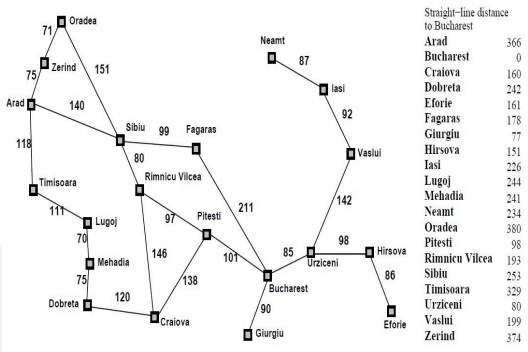




Example (optional)

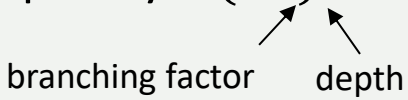






RBFS Summary

- Optimal with admissible heuristic function $h(n)$.
- Space complexity $O(bd)$.



- Time complexity difficult to analyze, depending on
  - accuracy of $h(n)$
  - how often the best path changes
- Slightly more efficient than IDA*.
- Both IDA* and RBFS suffering from using too little memory and may explore the same state multiple times.

6/6. More about heuristics

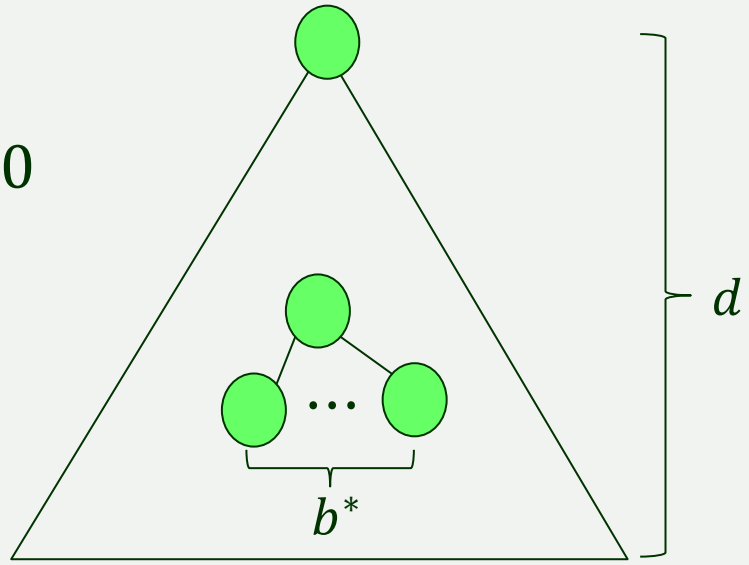
Quality of a heuristic is often measured by the effective branching factor.

If A* generates N nodes to find a solution at depth d, then its *effective branching factor* b^* is the root of the following equation:

$$N + 1 = 1 + b^* + (b^*)^2 + \cdots + (b^*)^d$$
$$= \left(1 - (b^*)^{d+1}\right)/(1 - b^*)$$

$$\Longrightarrow \quad (b^*)^{d+1} - (N+1)b^* + N = 0$$

Intuitively, the $N + 1$ nodes handled by A* would fill a tree of height d in which every node at depth $< d$ has exactly b^*children.



e.g. A* finds a solution at depth 5 using 52 nodes has $b^* = 1.92$.

Performance Comparison on 8-Puzzle

	Search Cost (nodes generated)			Effective Branching Factor		
d	BFS	A*(h_1)	A*(h_2)	BFS	A*(h_1)	A*(h_2)
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

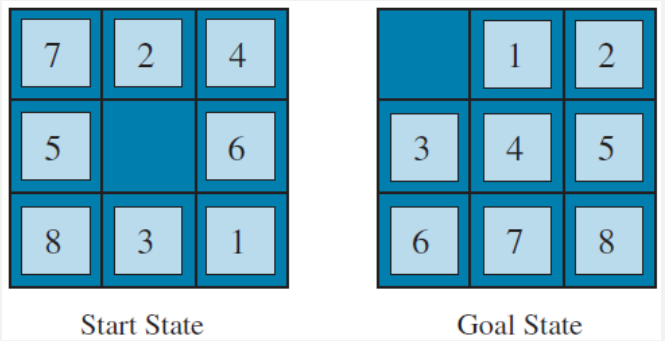
Figure 3.26 Comparison of the search costs and effective branching factors for 8-puzzle problems using breadth-first search, A* with h_1 (misplaced tiles), and A* with h_2 (Manhattan distance). Data are averaged over 100 puzzles for each solution length d from 6 to 28.

Good heuristics

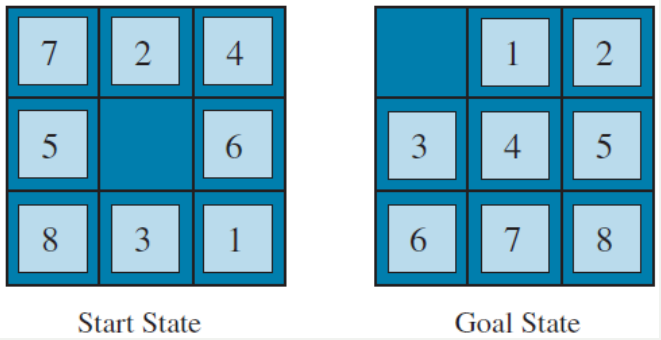
- Good heuristic: b^* is close to 1
- Good heuristic: domination
  - h_2 dominates h_1 if $h_1(n) \leq h_2(n) \leq h^*(n)$ for all n
  - The closer a heuristic is to h^*, the fewer nodes it will expand
  - Less time and space

Heuristics from relaxation

- The 8-puzzle problem



  - Tile can move from square A to square B
    - if A and B are horizontally or vertically adjacent, and
    - if B is blank
  - Relaxed versions: tile can move from A to B
    - if A and B are adjacent
    - if B is blank
    - always

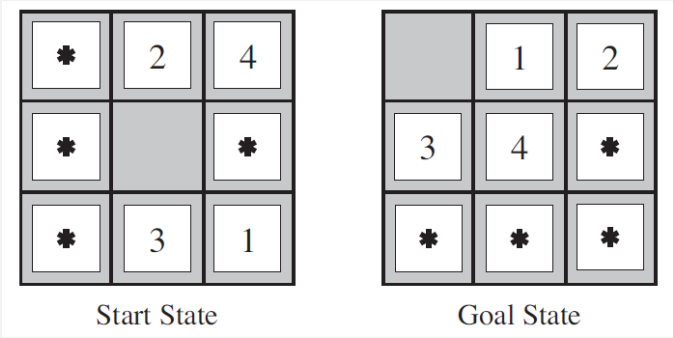


- Can always move from A to B
  - Optimal cost = # of tiles out of place
  - $h_1 = 8$
- Can move from A to B if adjacent
  - Optimal cost = Manhattan distance of all tiles from their goal
  - $h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$
  - true cost = 26

- If $h_2(n) \geq h_1(n)$ for all n (both admissible), then h_2 dominates h_1
- h_2 is better for search: it is guaranteed to expand less or equal # of nodes.
- Typical search costs (average number of nodes expanded):
- *d=12* ➔ *BFS* = 2,672 nodes, $A^*(h_1)$ = 279 nodes ,$A^*(h_2)$ = 84 nodes
- *d=24* ➔ *BFS* = too many nodes, $A^*(h_1)$ = 53,039 nodes, $A^*(h_2)$ = 5,733 nodes

Heuristics from subproblems

- A subproblem is part of the overall problem that must be solved
  - 1-2-3-4 or 5-6-7-8
  - or anything 2-4-6-8
  - Different from relaxation



- Pattern database approach: store the exact costs of every possible subproblem instance
  ➔ then compute an admissible heuristic h_{DB} for each complete state encountered during a search simply by looking up the corresponding subproblem configuration in the DB

Combining heuristics

- What if you have 2 heuristics, and neither one dominates?
- Combine: $h(n)=\max\{h_1(n),h_2(n)\}$
- Example
  - h_1= 1-2-3-4 subproblem
  - h_2= 5-6-7-8 subproblem
- Can generalize to any number of heuristics
- Can we add? $h(n) = c_1 h_1(n) + c_2 h_2(n)$
  - *May be able to learn c's, without a guarantee*